

SQL (Part-2) — Detailed Study Guide

Transactions, ACID Properties, Advanced Joins, Subqueries, Views, Indexes & Stored Procedures

Author: Golam Maula Lincoln

GitHub Profile: <https://github.com/gmlincoln>

Visit GitHub Profile [🔗](https://github.com/gmlincoln)

1. Transactions & ACID Properties

A **Transaction** is a sequence of one or more SQL operations performed as a single logical unit of work. A transaction must either complete entirely or be completely aborted.

ACID Properties

- **Atomicity:** All statements in a transaction succeed, or none succeed (all-or-nothing execution).
- **Consistency:** Data transitions safely from one valid database state to another, preserving all schema rules and constraints.
- **Isolation:** Concurrent or parallel transactions execute independently without interfering with each other's uncommitted data.
- **Durability:** Once a transaction is committed, its changes are permanently saved in non-volatile storage, even during system crashes.

Autocommit Control

By default, database systems automatically commit every individual SQL statement. You can disable or enable autocommit mode explicitly:

```
-- Disable autocommit
SET autocommit = 0;

-- Enable autocommit
SET autocommit = 1;
```

Transaction Commands: START, COMMIT, ROLLBACK & SAVEPOINT

1. **Start & Commit:** Commits the changes permanently if all updates succeed.

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 50 WHERE id = 1;
UPDATE accounts SET balance = balance + 50 WHERE id = 2;
COMMIT;
```

2. **Rollback:** Reverts all operations executed since the start of the transaction.

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 3;
ROLLBACK; -- Reverts both updates
```

3. **Savepoints:** Defines intermediate checkpoints within a transaction to perform partial rollbacks.

```

START TRANSACTION;
UPDATE accounts SET balance = balance + 1000 WHERE id = 1;
SAVEPOINT after_wallet_topup;

UPDATE accounts SET balance = balance + 10 WHERE id = 1;
ROLLBACK TO after_wallet_topup; -- Rolls back only the +10 update
COMMIT;                       -- Commits the +1000 topup

```

2. SQL Relational Joins

JOINS combine rows from two or more tables based on a logical relationship between shared key columns.

Sample Tables Reference

customers Table

customer_id	name	city
1	Alice	Mumbai
2	Bob	Delhi
3	Charlie	Bangalore
4	David	Mumbai

orders Table

order_id	customer_id	amount
101	1	500
102	1	900
103	2	300
104	5	700

1) INNER JOIN

Returns records that have matching values in both tables.

```

SELECT c.name, o.order_id, o.amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;

```

2) LEFT JOIN

Returns all records from the left table (`tableA`) along with matched records from the right table (`tableB`). Unmatched right rows contain `NULL`s.

```

SELECT *
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;

```

3) RIGHT JOIN

Returns all records from the right table (`tableB`) along with matched records from the left table (`tableA`). Unmatched left rows contain `NULL`s.

```

SELECT *
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;

```

4) FULL OUTER JOIN (MySQL Union Approach)

Returns all records when there is a match in either left or right table. In MySQL, FULL OUTER JOIN is implemented using `UNION` of `LEFT JOIN` and `RIGHT JOIN`.

```
SELECT * FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
UNION
SELECT * FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;
```

5) CROSS JOIN

Produces a Cartesian product combining each row from `tableA` with every row from `tableB`.

```
SELECT * FROM customers c CROSS JOIN orders o;
```

6) SELF JOIN

A regular join in which a table is joined with itself using distinct table aliases.

```
SELECT *
FROM customers A
JOIN customers B ON A.customer_id = B.customer_id;
```

7) Exclusive Joins (Left & Right Exclusive)

Left Exclusive Join: Returns records present *only* in the left table and missing in the right table.

```
SELECT * FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;
```

Right Exclusive Join: Returns records present *only* in the right table and missing in the left table.

```
SELECT * FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;
```

3. SQL Subqueries (Nested Queries)

A **Subquery** (or Inner/Nested Query) is a `SELECT` statement nested inside another outer query. It involves two or more `SELECT` statements.

1) Subquery in WHERE Clause

Filters rows based on an evaluated expression from the inner subquery.

```
SELECT *
FROM orders
WHERE amount > (
    SELECT AVG(amount) FROM orders
);
```

2) Subquery in SELECT Clause

Evaluates a scalar calculation or row-by-row count alongside selected fields.

```
SELECT c.name, (
    SELECT COUNT(*)
    FROM orders o
    WHERE o.customer_id = c.customer_id
) AS order_count
FROM customers c;
```

3) Subquery in FROM Clause (Derived Tables)

Treats the result set of an inner query as a temporary table (must specify a table alias).

```
SELECT summary.customer_id, summary.avg_amount
FROM (
    SELECT customer_id, AVG(amount) AS avg_amount
    FROM orders
    GROUP BY customer_id
) AS summary;
```

4. Views in SQL

A **View** is a virtual table based on the result-set of an SQL statement. It contains rows and columns just like a real table, but does not store data physically on disk (unless instantiated as a materialized view).

Syntax & Basic Operations

```
-- Creating a View
CREATE VIEW view1 AS
SELECT col1, col2 FROM table_name;

-- Querying a View
SELECT * FROM view1;
```

Key Characteristics of Views:

- **Dynamic Up-to-Date Data:** The database engine recreates the view every time a user queries it.
- **Zero Physical Storage:** No data is stored physically (it only stores the query definition).
- **Multi-table Integration:** Views can join and consolidate columns from multiple underlying tables.
- **Enhanced Security:** Protects sensitive data by exposing only specific columns to unauthorized users.

5. Indexes in SQL

Indexes are special database lookup tables used by the database search engine to speed up data retrieval. They function similarly to an index in a book.

Syntax & Operations

```
-- Single-column Index
CREATE INDEX idx_name ON table_name(col);

-- Composite (Multi-column) Index
CREATE INDEX idx_name ON table_name(col1, col2);

-- Show existing Indexes
SHOW INDEX FROM table_name;

-- Drop an Index
DROP INDEX idx_name ON table_name;
```

6. Stored Procedures

A **Stored Procedure** is a prepared, reusable set of SQL statements saved in the database catalog. It helps modularize code, improve performance, and restrict direct database manipulation.

Syntax & Implementation Example

```
-- Defining a Stored Procedure with IN and OUT parameters
DELIMITER $$
CREATE PROCEDURE check_balance (
    IN acc_id INT,
    OUT bal DECIMAL(10, 2)
)
BEGIN
    SELECT balance INTO bal
    FROM bank_accounts AS b
    WHERE b.account_id = acc_id;
END $$
DELIMITER ;

-- Calling the Stored Procedure
CALL check_balance(2, @balance);
SELECT @balance;

-- Dropping a Stored Procedure
DROP PROCEDURE IF EXISTS procedure_name;
```

✨ Keep Learning & Keep Exploring! ✨

Document created for GitHub profile: [gmlincoln \(Golam Maula Lincoln\)](#)